* Time Complexity -> Asymptotic Notation: Asymptotic notation the gro... rate of running time.
* Asymptotic Notation -> efficiency: It helps compare algorithm & determine which is more efficient.

Big-O, Big-Theta, Big-omega:

* Big-O (O): upper bound (worst case)
* Big-Theta (Θ): Exact bound (Average / Exact growth)
* Big-omega (Ω): Lower bound (Best Case).

Justification:-

→ Those notation classify how an algorithm grows for large value of n, making comparisons independent of hardware.

Interpretation

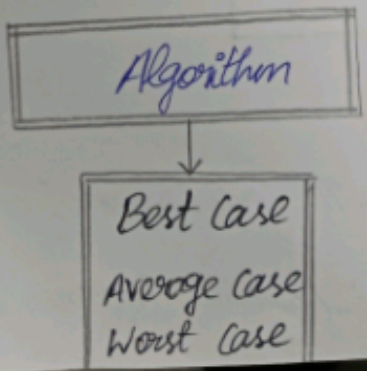
→ The completed map shows the complete process of evaluating an algorithm from its input size to measuring its efficiency in a structure manner.

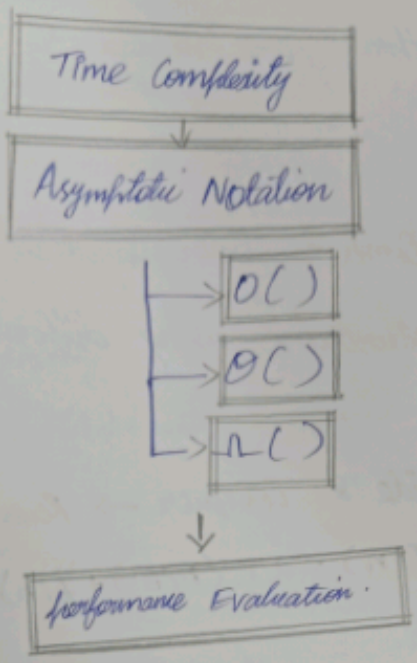
---

2) Concept Map: Algorithm → Best Case, Average Case, Worst-case

→ Time Complexity → performance Evaluation

Concept Map





Explanation:

* Best Case : Minimum execution time.
* Average Case : Expected execution time.
* Worst Case : Maximum execution time.
* These three cases determine the time complexity.
* The Time Complexity is represented using asymptotic notation.
* finally, performance is evaluated.

Justification

Considering only one case may not represent actual performance.

* Best Case is optimistic.
* Worst case guarantees performance.
* Average case represents practical execution

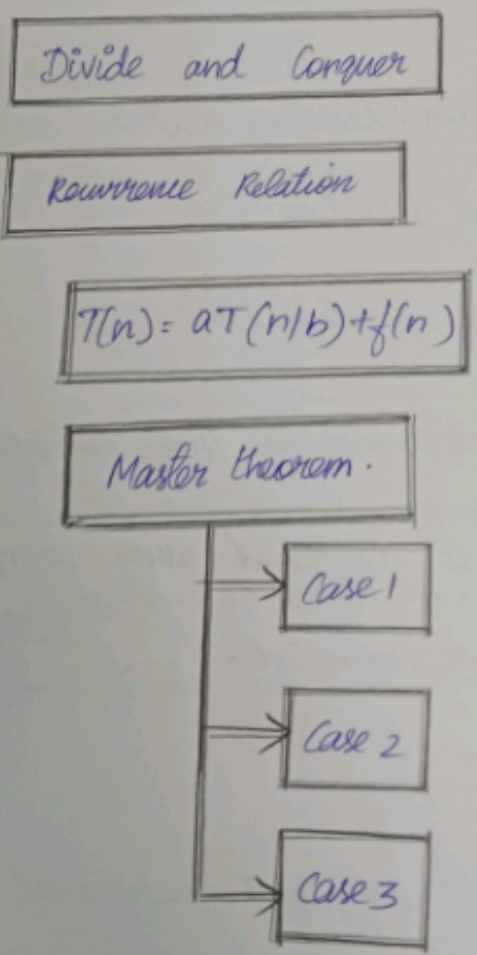
∴ All three are needed.

Interpretation:-

The map helps compare both practical and theoretical performance of algorithms using different execution scenarios.

(13). Concept map: Divide & Conquer → Recurrence Relation → $T(n) = aT(n/b) + f(n)$ → Master theorem.

Concept Map:-



Cases of Master Theorem :-

Case 1

$f(n) < n^{\log_b(a)}$

Time Complexity

$O(n^{\log_b(a)})$

Case 2

$f(n) = n^{\log_b(a)}$

Time Complexity

$O(n^{\log_b(a)} \log n)$

Case 3

$f(n) > n^{\log_b(a)}$

Time complexity : $O(f(n))$

Explanation :

* Divide & conquer divides a problem into smaller parts
* This creates a recurrence relation.
* Most divide and - conquer recurrences follow the form.

$$T(n) = aT(n/b) + f(n)$$

* Master theorem Solves the recurrence by comparing $f(n)$ with $n^{\log_b(a)}$

Justification

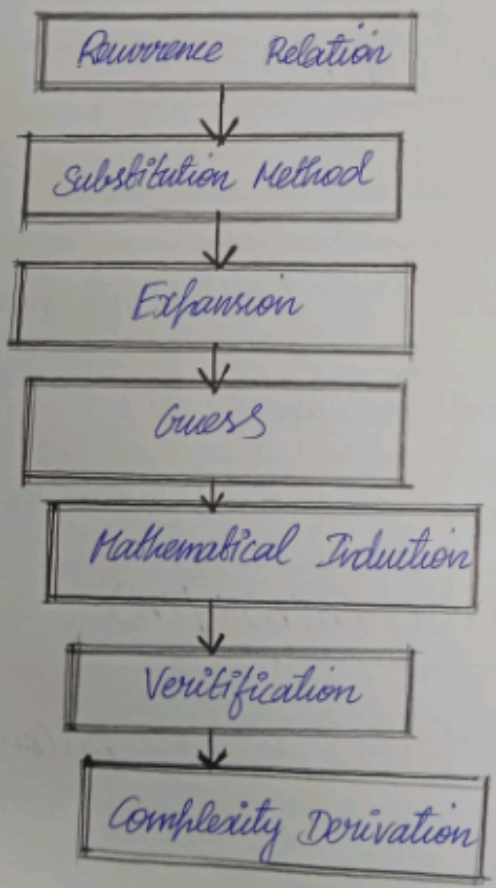
The Master theorem avoids lengthy calculations & directly provides the time complexity.

Interpretation

The completed map provides a systematic method for analyzing divide-and-conquer algorithms efficiently.

24) Concept Map : Rewrrence Relation → Substitution Method → Expansion → Guess → Induction → Verification → Complexity Derivation :-

Concept Map :-



Explanation

* Start with the recurrence relation.
* Expand the recurrence relation.
* Observe the pattern.
* Guess the final complexity.
* Use mathematical induction to prove the guess.
* Verify that the assumption is correct.
* Obtain the final time complexity.

Justification

=> Verification ensures that the guessed solution satisfies the original recurrence, making the proof correct.

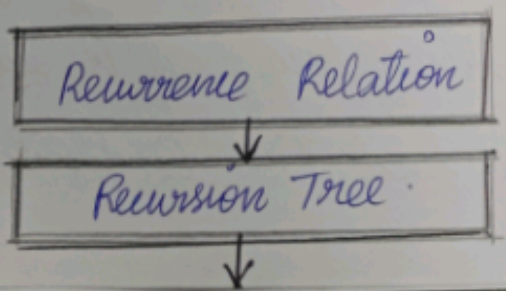
Interpretation

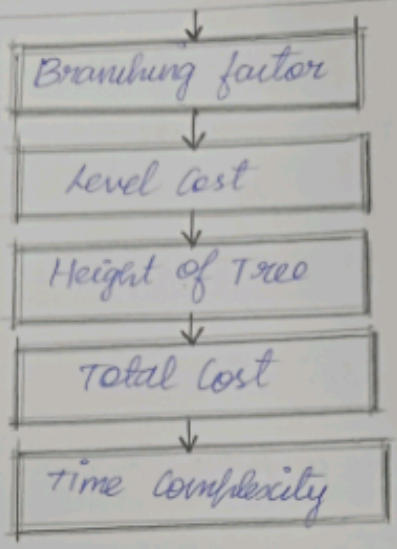
=> This concept map provides a clear step-step framework for solving recurrence relations systematically.

---

⑤ Concept Map : Recurrence Relation → Recursion Tree → Level Cost → Height of Tree → Total Cost → Time Complexity

Concept map:-





Explanation :

* A recurrence relation is represented as a recursion tree.
* Each node represents a recursive call.
* The branching factor shows how many child calls are generated.
* Every level has its own computation cost (Level cost)
* The height represents the ~~nber~~ number of recursive levels.
* Summing the costs of all levels gives the total cost.
* The total cost determines all levels gives the total cost.

Justification

⇒ Adding the costs of every level provides the complete running time of the recursive algorithm.

Interpretation

⇒ The recursion tree makes recursive complexity analysis visual, helping students understand how work is distributed across recursive calls and derive the final time complexity easily.